

SEMANA 2 · TUTORÍA SINCRÓNICA

Modelos de Aprendizaje Supervisado

Sábado 26/09/2026 · 3 h MIAR0525 · Maestría en Inteligencia Artificial

Agenda de hoy

Tiempo	Bloque	Animaciones
0:00–0:10	Repaso de la semana 1: errores frecuentes	—
0:10–0:50	Regresión lineal, descenso de gradiente y regularización	A2.1 · A2.2
0:50–1:25	Regresión logística y calibración	A2.3 · A2.4
1:25–1:35	Pausa	—
1:35–2:15	SVM y k vecinos: margen, kernels y distancias	A2.5 · A2.11
2:15–2:45	Árboles, bosques y boosting	A2.6 · A2.7 · A2.8
2:45–3:00	Comparar con honestidad y Tarea 2	A2.9 · A2.10

Tres errores que conviene no repetir

- 1 **Preprocesar antes de partir.** El escalador o el imputador se ajustan con todos los datos: fuga. Todo va dentro del Pipeline .
- 2 **Reportar exactitud con clases desbalanceadas.** Con 74 % de "no abandona", 76 % no dice nada. Baseline, recall y PR-AUC.
- 3 **Umbral 0.5 por defecto.** El umbral se elige con la matriz de costos y por validación cruzada, nunca con la prueba.

Tres familias de modelos y una disciplina

Lineales

Regresión lineal y logística, con Ridge y Lasso. Rápidos, interpretables, pocas sorpresas.

SVM y k vecinos

El margen máximo y el truco del kernel; KNN, que decide por semejanza con los casos guardados.

Árboles y ensambles

Reglas legibles; bosques que promedian y boosting que corrige.

La disciplina: comparar con honestidad

Pipeline, búsqueda de hiperparámetros sin fuga, costos, media $\pm$ DE y experimentos registrados en MLflow.

BLOQUE 1 · 0:10–0:50

Regresión lineal y regularización

Cómo aprende un modelo lineal y cómo evitar que ajuste el ruido.

Una recta (o un hiperplano) y su error

$$\hat{y} = X\beta \quad \hat{\beta} = \arg \min_{\beta} \frac{1}{n} \|y - X\beta\|^2$$

- El ECM es un **cuenco convexo**: un único mínimo.
- β_j : cambio esperado en y por una unidad de x_j , **con las demás fijas**.
- Con variables correlacionadas, los coeficientes individuales se vuelven inestables.

Pregunta

Si estandarizamos las variables, ¿cambian las predicciones? ¿Y los coeficientes?

Dos caminos al mismo mínimo

Ecuación normal

$$X^T X \hat{\beta} = X^T y$$

Exacta. Costo $\propto p^3$. Es lo que hace
LinearRegression (con QR o SVD).

Descenso de gradiente

$$\beta \leftarrow \beta - \eta \frac{2}{n} X^T (X\beta - y)$$

Iterativo. Escala a millones de filas con
 mini-lotes (**SGDRegressor**).

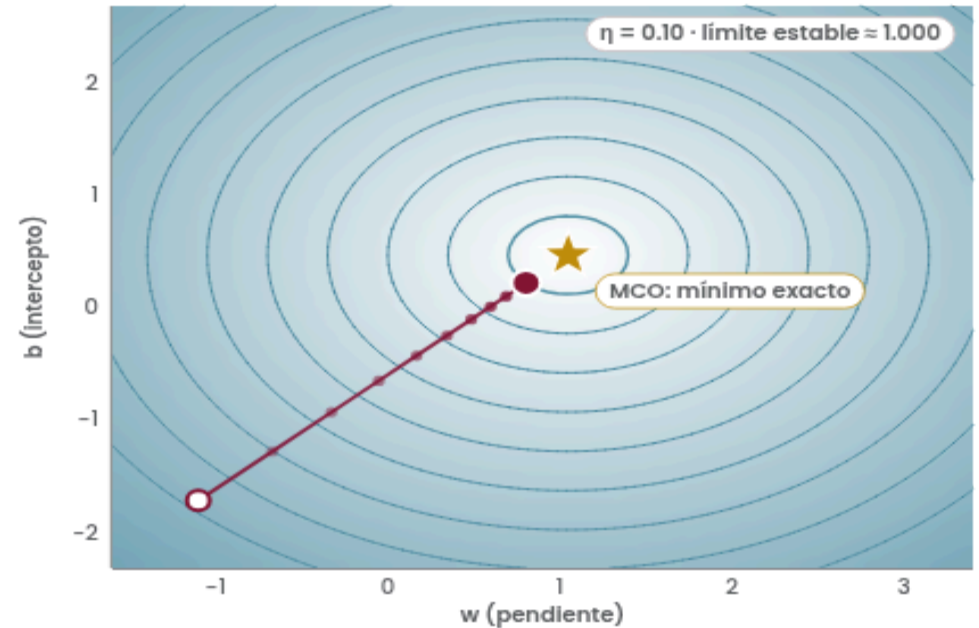
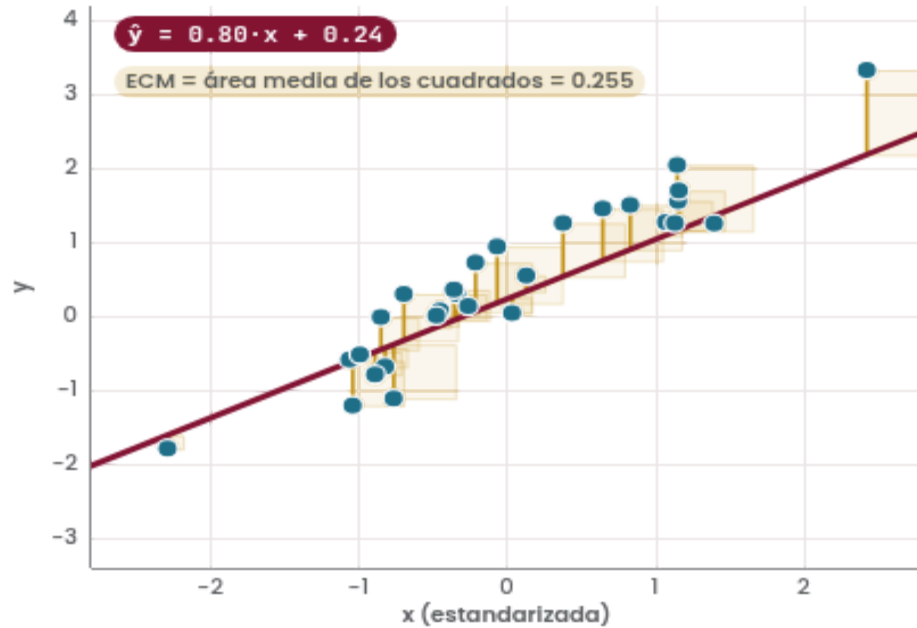
Python

```
Xb = np.column_stack([np.ones(len(X)), X])
beta = np.linalg.solve(Xb.T @ Xb, Xb.T @ y)

beta_gd = np.zeros(Xb.shape[1])
for _ in range(500):
    grad = 2 / len(y) * Xb.T @ (Xb @ beta_gd - y)
    beta_gd -= 0.1 * grad          #  $\eta = 0.1$ 

np.allclose(beta, beta_gd)      # True (x estandarizada)
```

Bajar por la superficie de pérdida



Ni muy lenta ni por encima del límite

- Si $\eta > 2/\lambda_{\max}$ del hessiano $\frac{2}{n}X^{\top}X$, cada paso se pasa del mínimo: **diverge**.
- Variables en escalas muy distintas estiran el cuenco: la tasa estable queda **diminuta**.
- En E2.1, con x_2 50 veces más grande, $\eta = 8 \cdot 10^{-4}$ ya diverge; estandarizada, $\eta = 0.1$ converge en ≈ 20 pasos.

Regla

Estandariza dentro del pipeline antes de cualquier método por gradiente, SVM, kNN o regularización.

Castigar el tamaño de los coeficientes

$$\text{Ridge: } \|y - X\beta\|^2 + \alpha\|\beta\|_2^2 \quad \text{Lasso: } \frac{1}{2n}\|y - X\beta\|^2 + \alpha\|\beta\|_1$$

Ridge (L2)

Encoge todos los coeficientes; no anula ninguno. Útil con variables correlacionadas.

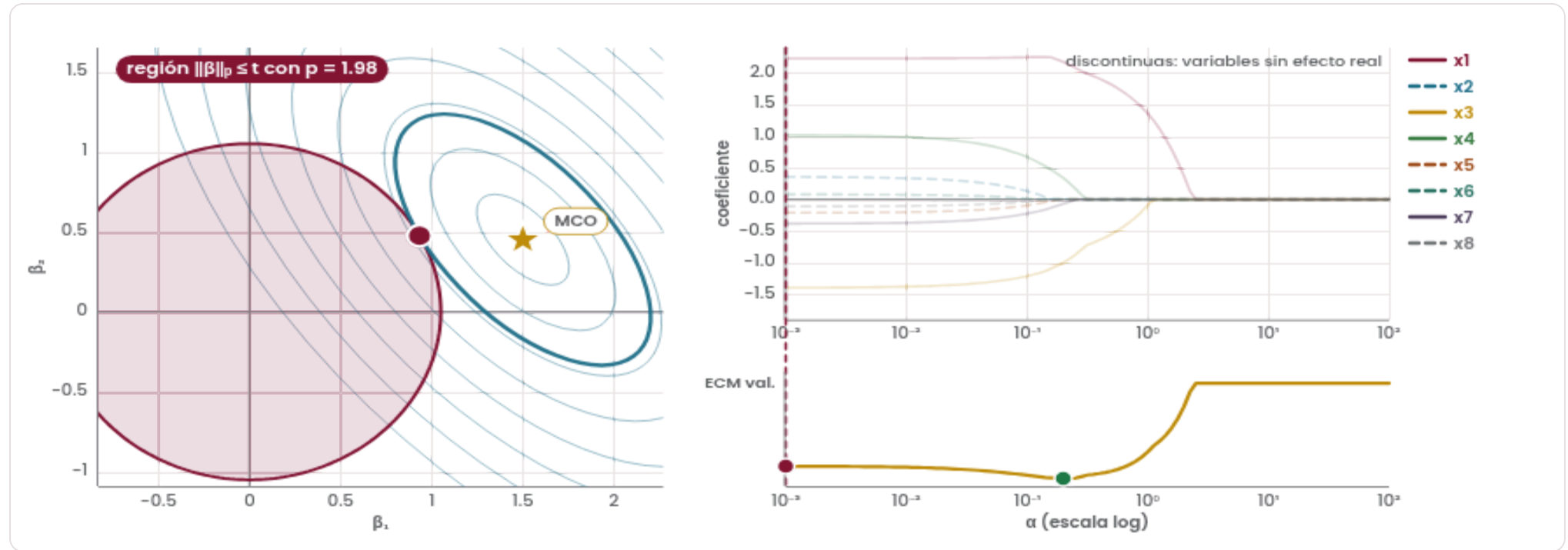
Lasso (L1)

Anula coeficientes:
selecciona variables.

Elastic Net

Mezcla ambas; con variables correlacionadas las conserva en grupo.

¿Por qué Lasso produce ceros exactos?



Regularizar ayuda contra la varianza, no contra el sesgo

- Con 6 variables de ruido y 200 filas, Lasso anula 5 de 6 y conserva las dos reales.
- En California Housing (8 variables, 16 mil filas), $MAE \approx 0.53$ **con y sin** regularización: el problema es el sesgo.
- Lo que sí baja el error es dar flexibilidad: con interacciones de grado 2, $MAE \approx 0.48$.

Python

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV, LassoCV

ridge = make_pipeline(StandardScaler(),
                      RidgeCV(alphas=np.logspace(-3, 3, 30)))
lasso = make_pipeline(StandardScaler(),
                      LassoCV(alphas=np.logspace(-4, 0, 30), cv=5))
```

BLOQUE 2 · 0:50–1:25

Regresión logística y calibración

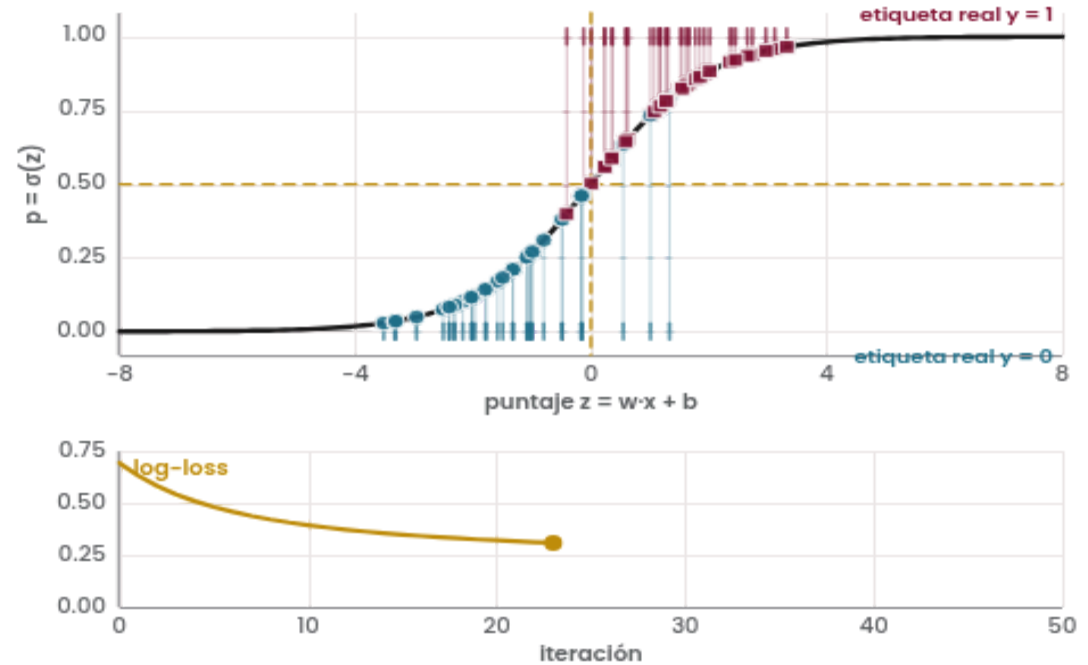
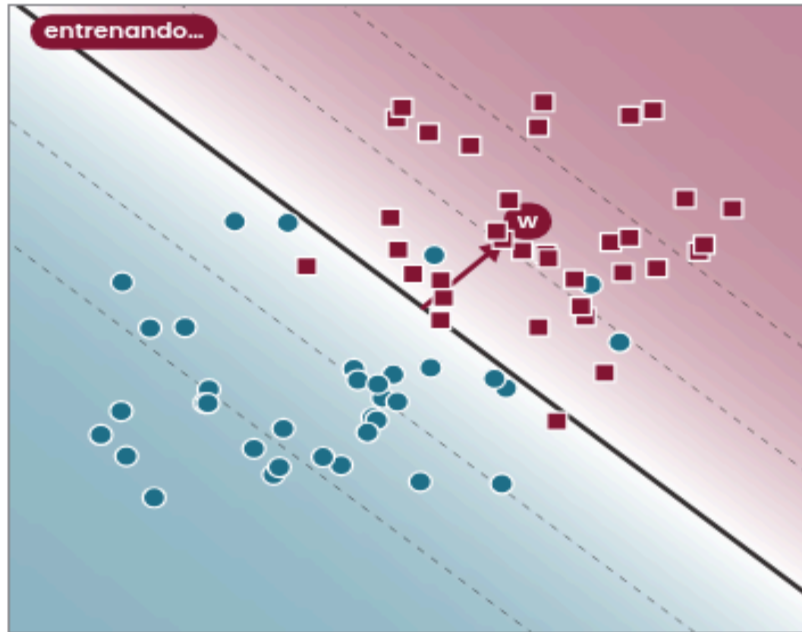
De un puntaje lineal a una probabilidad en la que se pueda confiar.

Un puntaje lineal pasado por la sigmoide

$$z = w^\top x + b \quad p(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

- La frontera $p = 0.5$ es un **hiperplano**: es un clasificador lineal.
- z es proporcional a la distancia con signo a la frontera.
- e^{w_j} : factor que multiplica las chances $p/(1 - p)$ por cada unidad de x_j .

Entrenar es mover la frontera



La log-loss castiga la confianza equivocada

$$L = -\frac{1}{n} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Acierta con 0.9

$-\log 0.9 = \mathbf{0.11}$

Duda: 0.5

$-\log 0.5 = \mathbf{0.69}$

**Se equivoca
con 0.95**

$-\log 0.05 = \mathbf{3.00}$

C regula; class_weight reequilibra

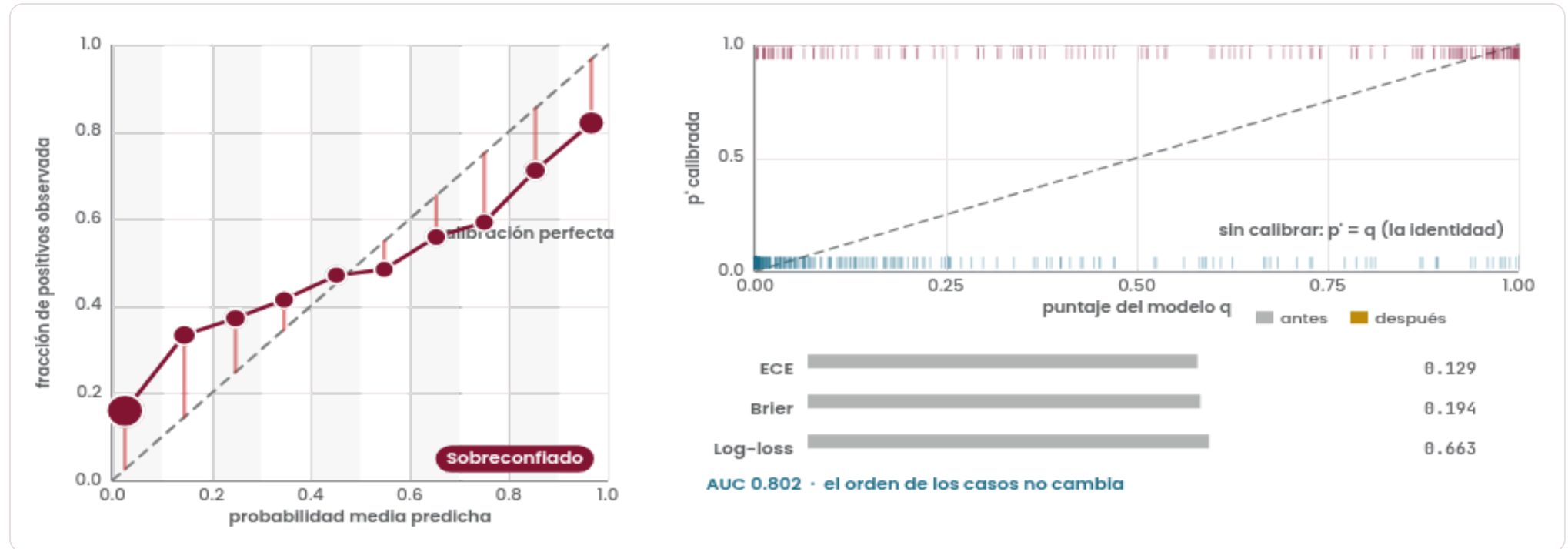
$$C = 1/\lambda$$

- Por defecto $C = 1$ (regularización L2).
- Menos $C \rightarrow \|w\|$ más chico, probabilidades más cerca de 0.5.
- Sin regularización y clases separables, $\|w\|$ crece sin fin.

`class_weight="balanced"`

- Sube el recall con umbral 0.5.
- **Desplaza las probabilidades:** en Bank Marketing (11.7 % positivos) la probabilidad media predicha pasa a 0.42.
- El orden (AUC) casi no cambia.

Si dice 0.8, ¿aciertan 8 de cada 10?



Llamar si $8p - 1 > 0$, es decir, si $p > 0.125$

Modelo (Bank Marketing, prueba)	Llamadas con $p \geq 0.125$	Beneficio
Logística sin pesos (calibrada)	2 939	3 637
Logística balanceada (sin calibrar)	11 010	-490
Balanceada + Platt	—	3 701

La fórmula del valor esperado solo funciona con **probabilidades calibradas**.

1:25–1:35

Pausa · 10 minutos

Al volver: ¿cuál de todas las fronteras que separan es la mejor?

BLOQUE 3 · 1:35–2:15

SVM y k vecinos

El margen máximo, el truco del kernel y decidir por semejanza.

La frontera con la calle más ancha

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y_i(w^\top x_i + b))$$

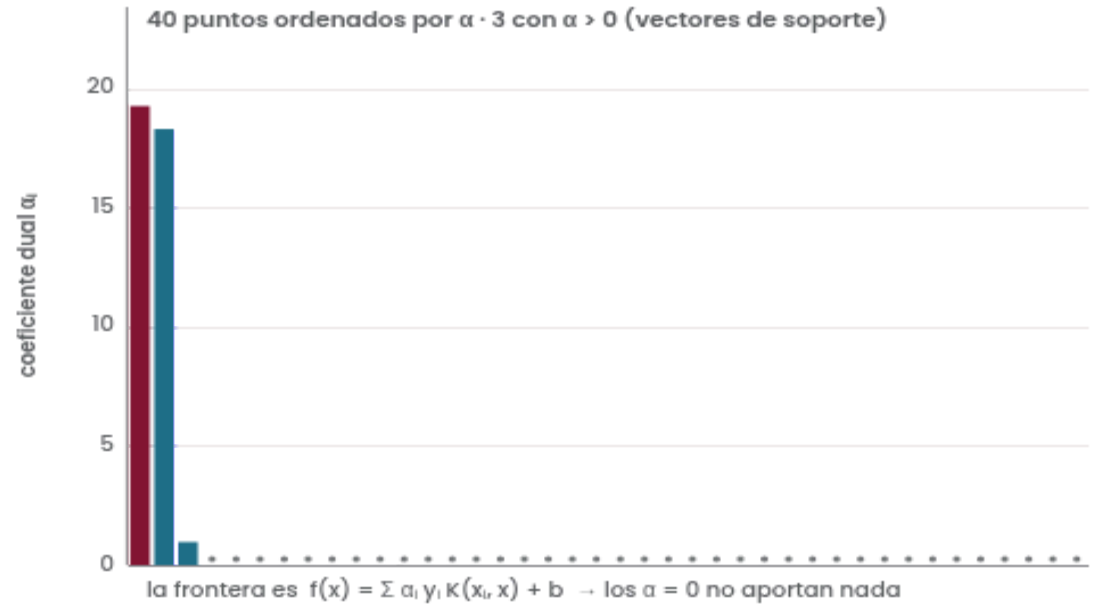
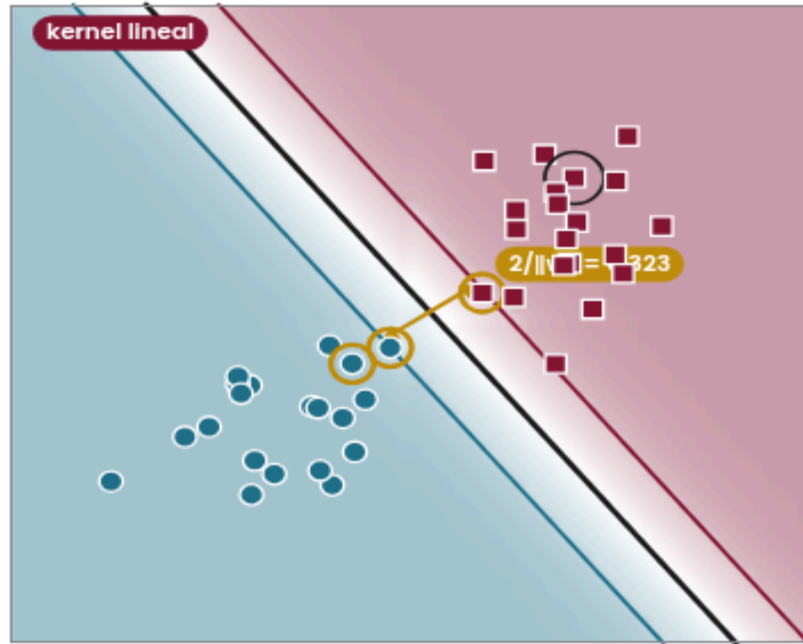
- Ancho de la calle: $2/\|w\|$.
- La **hinge** vale 0 fuera del margen: esos puntos no influyen.
- $f(x) = \sum_i \alpha_i y_i K(x_i, x) + b$, con $\alpha_i > 0$ solo en los **vectores de soporte**.

C

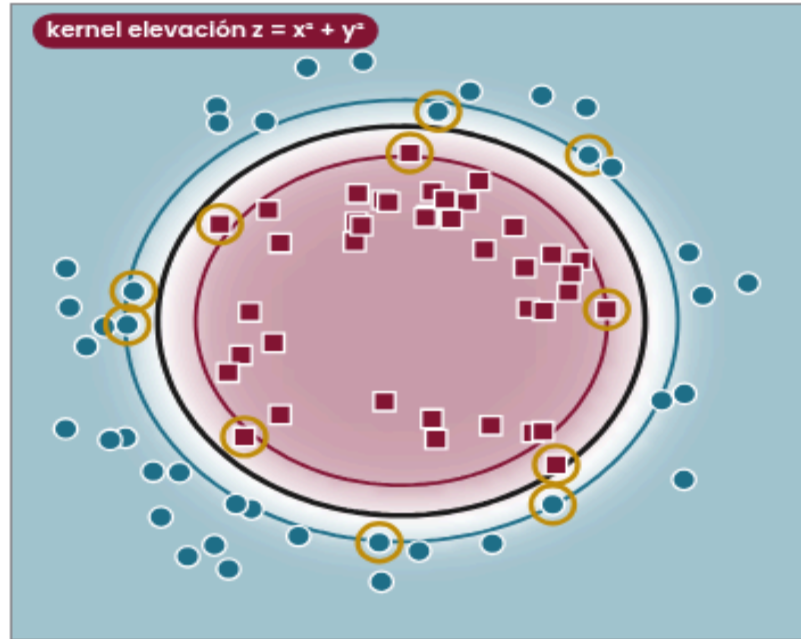
Grande: pocas violaciones, calle angosta, riesgo de sobreajuste.

Pequeño: calle ancha, más errores; muy pequeño, subajuste.

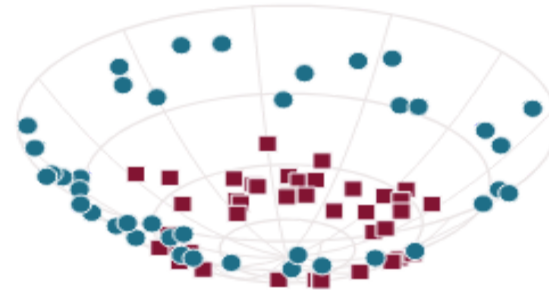
Solo el borde define la frontera



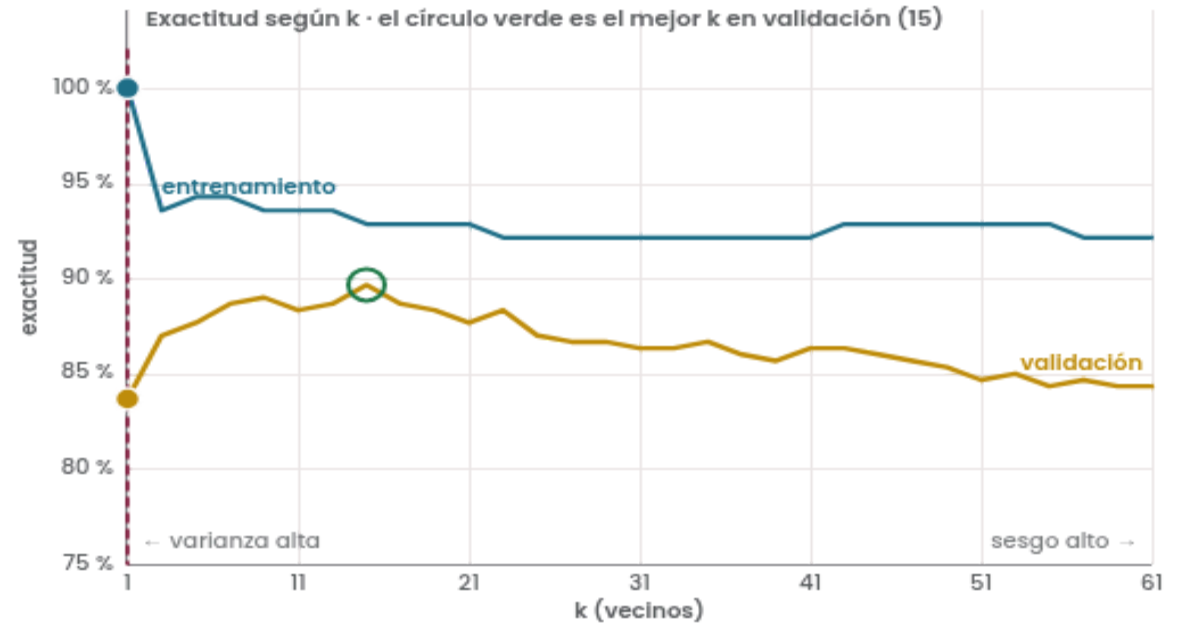
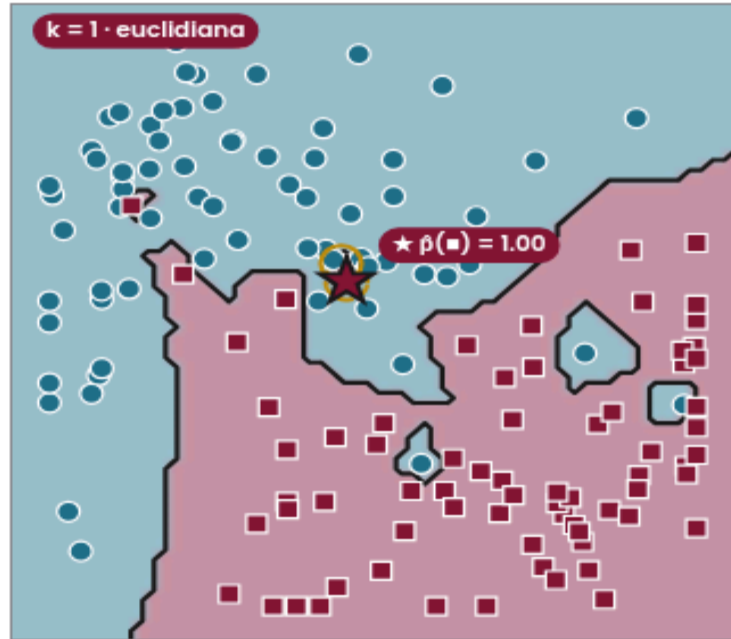
En 3D un plano separa lo que en 2D pedía un círculo



elevando: $z = x^2 + y^2 \dots$



Deciden los k vecinos más cercanos



Todo depende de la distancia

- **k** se elige por validación cruzada: en lunas, 0.857 con $k = 1$ y 0.910 con el mejor k .
- **Escalar** es obligatorio: con una variable $\times 100$, 0.792 sin escalar y 0.895 con `StandardScaler`.
- Con `weights="distance"` la exactitud de entrenamiento es siempre 100 %: no sirve para diagnosticar.
- **Maldición de la dimensión**: con 100 variables de ruido todos parecen igual de lejos (A2.11: la exactitud cae de 90 % a 70 %).

Python

```
knn = make_pipeline(  
    StandardScaler(), KNeighborsClassifier()  
)  
malla = {  
    "kneighborsclassifier__n_neighbors":  
        [1, 5, 11, 21, 41],  
    "kneighborsclassifier__weights":  
        ["uniform", "distance"]  
}  
busq = GridSearchCV(knn, malla, cv=5,  
                    scoring="f1")
```

Detección de phishing con 30 indicadores

Modelo (prueba)	F1
Regresión logística (baseline)	0.916
SVC lineal	0.925
SVC RBF ($C = 30$, $\gamma = 0.03$)	0.960
KNN ($k = 21$, pesos $1/d$)	0.949

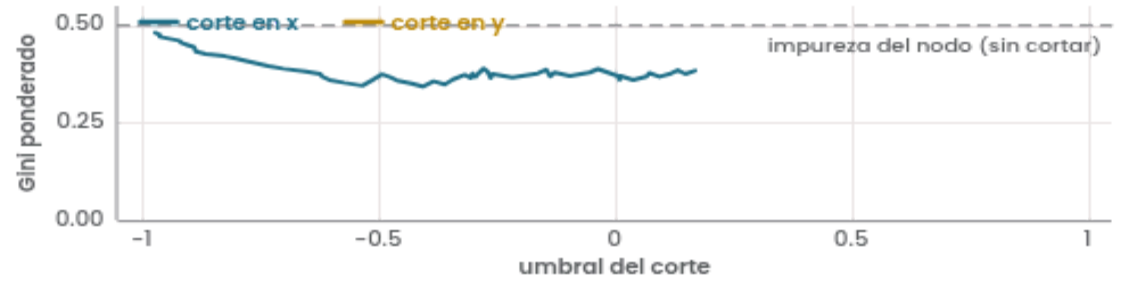
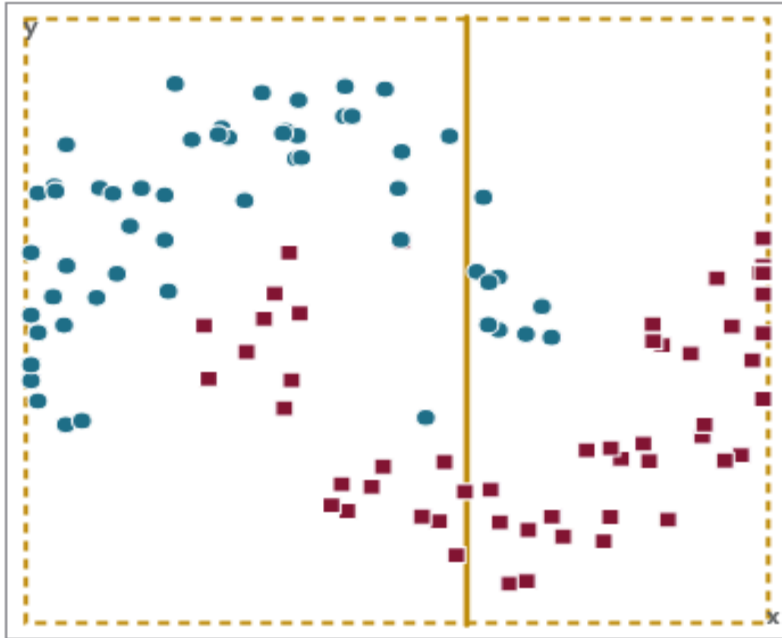
- C y γ se buscan juntos, en escala logarítmica, **solo con el entrenamiento**.
- El precio de la SVM: ≈ 1300 vectores de soporte que guardar y un costo que crece más que cuadráticamente con n .
- KNN "entrena" en milisegundos, pero guarda los 8 291 sitios y cada predicción los recorre.
- Con millones de filas: `LinearSVC`, `Nystroem` o árboles.

BLOQUE 4 · 2:15–2:45

Árboles, bosques y boosting

Preguntas voraces, votación y corrección secuencial.

La pregunta que deja los grupos más puros



[55, 55] · 110

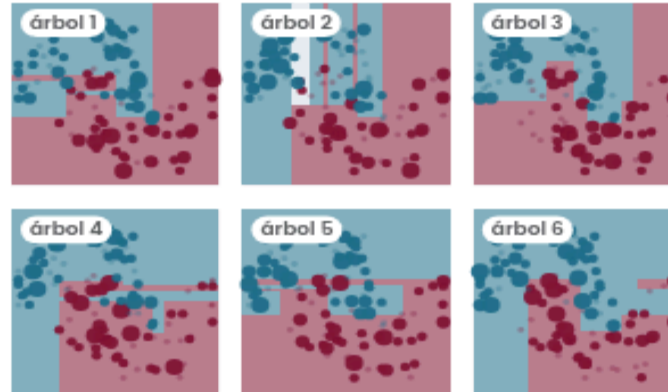
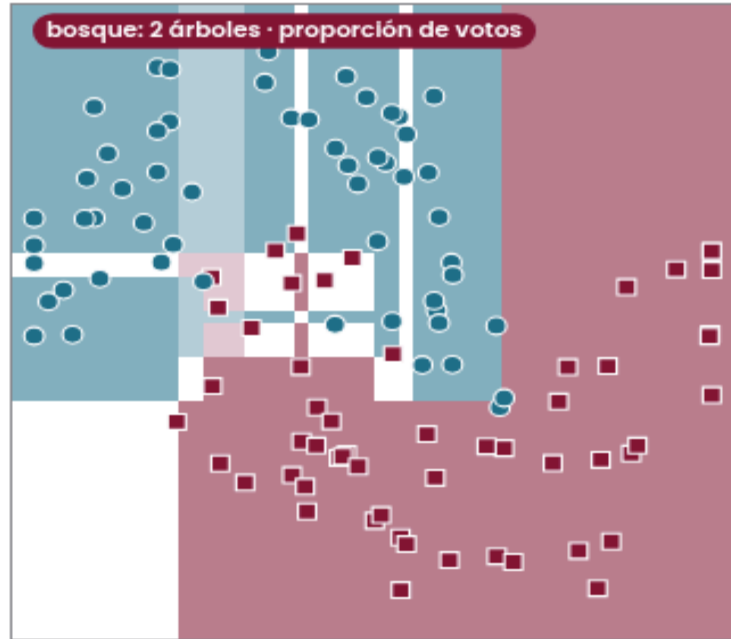
hojas: [clase 0, clase 1] · n

Gini, ganancia y el costo de la profundidad

$$G = 1 - \sum_k p_k^2 \quad \text{ganancia} = G(\text{padre}) - \left[\frac{n_I}{n} G_I + \frac{n_D}{n} G_D \right]$$

credit-g · max_depth	2	4	8	sin límite
AUC entrenamiento	0.72	0.82	0.96	1.00
AUC validación	0.67	0.68	0.64	0.60

Muchos árboles distintos votan



Validación gratis e importancias honestas

Error fuera de la bolsa

Exactitud OOB **0.757** frente a **0.747 ± 0.017** por validación cruzada.

Cada punto lo juzgan solo los árboles que no lo vieron.

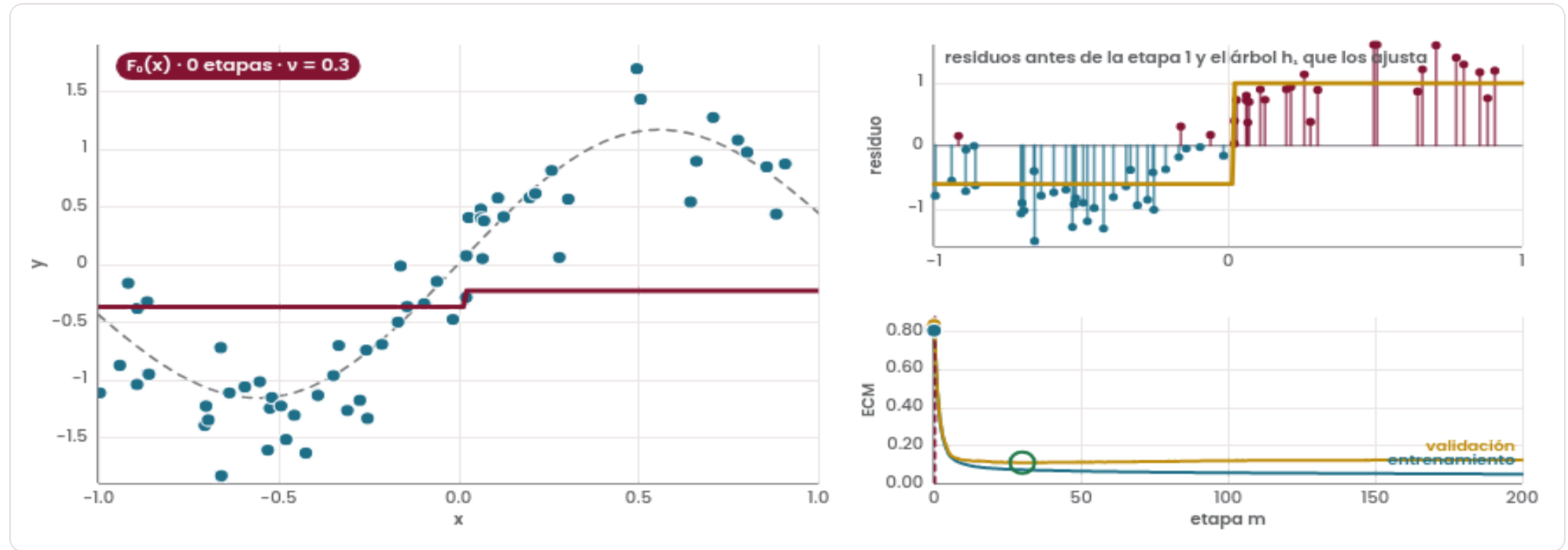
MDI frente a permutación

MDI (entrenamiento): `credit_amount`, `duration`, `age`.

Permutación (prueba):
`checking_status`, `duration`,
`credit_history`.

La MDI favorece a las variables con muchos valores.

Cada árbol corrige lo que falta



Promediar en paralelo o corregir en secuencia

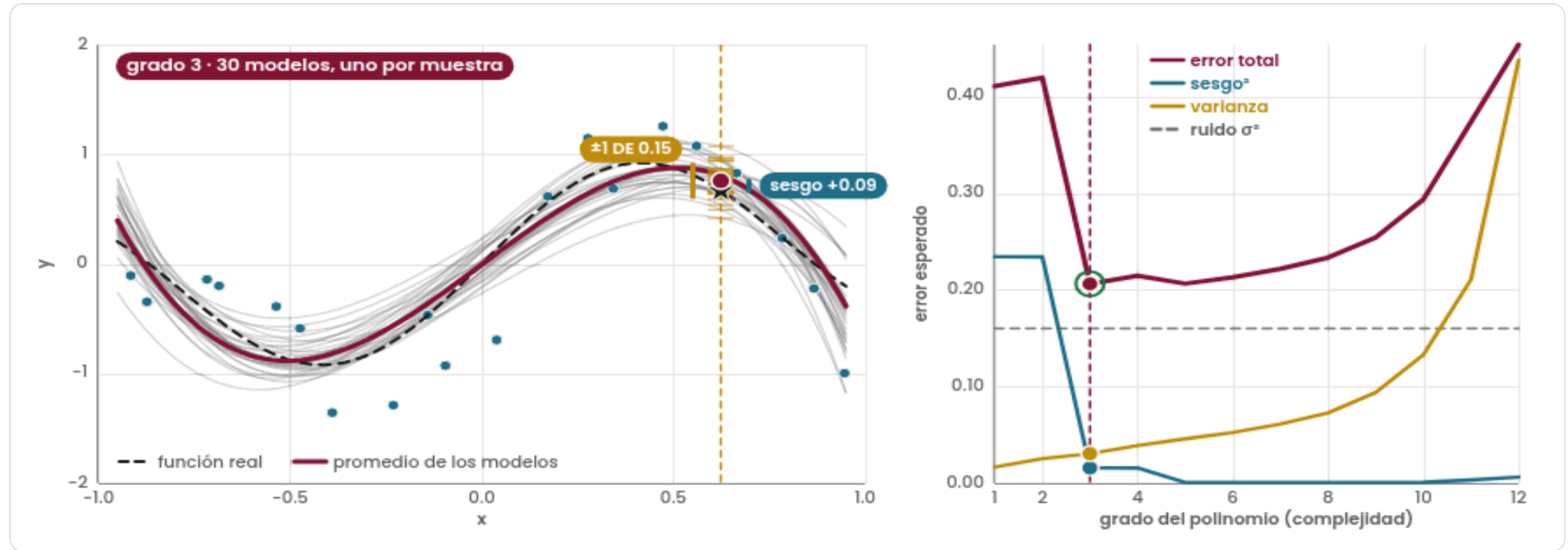
	Bagging / bosque aleatorio	Boosting (HistGradientBoosting)
Aprendices	Árboles profundos, independientes	Árboles pequeños, en secuencia
Reduce	Varianza	Sesgo (y algo de varianza)
¿Más aprendices sobreajusta?	No	Sí: parada temprana
Hiperparámetros clave	<code>max_features</code> , <code>min_samples_leaf</code>	<code>learning_rate</code> , <code>max_iter</code> , <code>max_depth</code>

BLOQUE 5 · 2:45–3:00

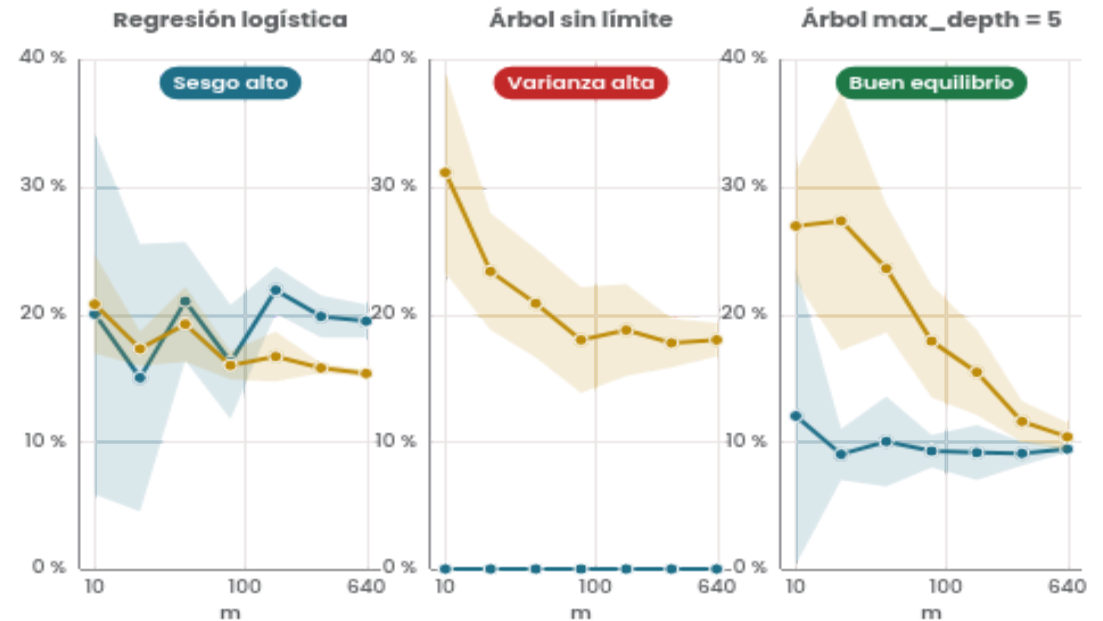
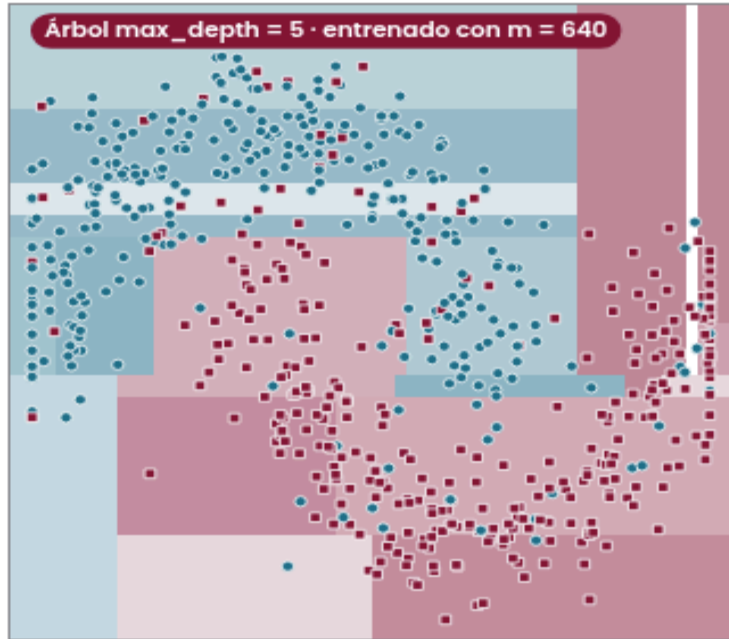
Comparar con honestidad

Las diferencias tienen que superar al ruido de la estimación.

Error = sesgo² + varianza + ruido



¿Más datos o más capacidad?



Cómo comparar modelos sin engañarse

- 1 Prueba separada al inicio; se usa **una vez**.
- 2 Todo el preprocesamiento dentro del pipeline.
- 3 Hiperparámetros con `GridSearchCV` solo con el entrenamiento.
- 4 Métrica de negocio (costo) y umbral por validación cruzada.
- 5 Media $\pm$ DE; ante empate, el modelo más simple.
- 6 Cada corrida, registrada en MLflow.

El optimismo de no anidar

Sobre **ruido puro** (AUC real 0.5), el mejor de una malla de 36 combinaciones reporta **0.54**; la validación anidada, **0.49**.

Es sesgo de selección: elegir y evaluar con los mismos folds.

Una corrida de MLflow por modelo

Python

```
import mlflow
from sklearn.model_selection import GridSearchCV

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("T2-jm1")
for nombre, (pipe, malla) in candidatos.items():
    with mlflow.start_run(run_name=nombre):
        busq = GridSearchCV(pipe, malla, cv=cv, scoring="average_precision")
        busq.fit(X_train, y_train)
        mlflow.log_params(busq.best_params_)
        mlflow.log_metrics({"pr_auc_cv": busq.best_score_, "costo_cv": costo_cv})
        mlflow.set_tags({"familia": nombre, "dataset": "jm1 (OpenML 1053)"})

# comparar: mlflow ui --backend-store-uri sqlite:///mlflow.db
# o en código: mlflow.search_runs(experiment_names=["T2-jm1"])
```

Cinco modelos, un costo 5:1

Modelo	AUC CV	Umbral por costo	Costo CV por solicitud
Bosque aleatorio	0.796 ± 0.019	0.17	0.512
Regresión logística	0.782 ± 0.024	0.14	0.523
SVM RBF	0.781 ± 0.028	0.17	0.532
HistGradientBoosting	0.778 ± 0.021	0.12	0.540
Árbol	0.696 ± 0.054	0.20	0.584

En la prueba: bosque con umbral 0.17 → **0.556** por solicitud; con 0.5 → 1.000, peor que rechazar a todos (0.700).

Comparación reproducible de modelos supervisados

- Datos: **jml** (OpenML 1053): 10 885 módulos de software, $\approx 19\%$ defectuosos, con duplicados.
- Limpieza justificada y prueba separada primero.
- Logística, SVM, árbol, bosque y HistGradientBoosting.
- Búsqueda sin fuga, desbalance (pesos y umbral).
- Curvas de aprendizaje y calibración del elegido.
- Todo registrado en MLflow; media $\pm$ DE; recomendación.
- IA agéntica o de chat permitida, **declarada**.

Rúbrica (100 pts)

Reproducibilidad y MLflow 15 · Limpieza y validación sin fuga 20 · Implementación y búsqueda 20 · Desbalance y umbral 10 · Análisis 20 · Recomendación y limitaciones 15

Antes de irte, responde en el chat

- 1 Tu SVM RBF tiene 100 % en entrenamiento y 81 % en validación. ¿Hacia dónde mueves C y gamma?
- 2 Dos modelos empatan en AUC, pero uno tiene mejor Brier. ¿Cuál usarías para decidir por valor esperado y por qué?

A CONTINUACIÓN · LABORATORIO GUIADO DE 1.5 H

Ahora, a comparar modelos de verdad

E2.4 · Nivel 3: cinco modelos + MLflow

Tarea 2 · vie 02/10